Patrones multivariados del tráfico de red y detección de ataques en UNSW-NB15

Proyecto integrador

[Integrante 1]
[Integrante 2]
[Integrante 3]
[Integrante 4]

Universidad Católica de Córdoba
Facultad de [completar]
Carrera de [completar]
Sede Río Cuarto

Materia: Estadística Multivariada
Docente a cargo: Prof. Silvana Malpassi

Fecha de entrega: [día] de [mes] de 2026

Índice

1. Introducción **4**

2. Datos **4**

3. Análisis de componentes principales **4**

4. Análisis factorial **4**

5. Análisis de conglomerados **4**

6. Análisis de correspondencias simple y múltiples **4**

7. Regresión logística **4**

8. Integración y conclusiones **4**

Referencias bibliográficas **5**

A. Figuras y tablas complementarias **6**

B. Código fuente **6**

B.1. Scripts de R 6

`R/00_datos.R` 6

`R/01_acp.R` 8

`R/02_af.R` 9

`R/aux_diagnostico_af.R` 10

`R/03_cluster.R` 11

`R/04_acm.R` 12

`R/05_integracion.R` 13

`R/06_logistica.R` 14

`R/ejecutar_todo.R` 17

B.2. Scripts de Python 17

`python/graficos_etapa0.py` 17

`python/graficos_parte2.py` 19

`python/graficos_logistica.py` 23

`python/verificar_resultados.py` 25

Índice de figuras

Índice de tablas

1. **Introducción**

   [Pendiente]

2. **Datos**

   [Pendiente]

3. **Análisis de componentes principales**

   [Pendiente]

4. **Análisis factorial**

   [Pendiente]

5. **Análisis de conglomerados**

   [Pendiente]

6. **Análisis de correspondencias simple y múltiples**

   [Pendiente]

7. **Regresión logística**

   [Pendiente]

8. **Integración y conclusiones**

   [Pendiente]

Referencias bibliográficas

A. Figuras y tablas complementarias

[Pendiente]

B. Código fuente

Los scripts se ejecutan desde la carpeta principal del repositorio. Primero se corre `R/ejecutar_todo.R`, que genera las tablas en `resultados/tablas`; luego los scripts de Python dibujan las figuras a partir de esas tablas. Todos los procesos aleatorios usan la semilla 42.

B.1. Scripts de R

`R/00_datos.R` · **Etapa 0: importación, calidad, descriptiva y supuestos**

```r
# ETAPA 0. Ejecutar desde la carpeta principal del proyecto.
# Instalar una sola vez: install.packages(c("readxl", "psych"))
# Ejecutar: source("R/00_datos.R", encoding = "UTF-8")
library(readxl)
library(psych)
set.seed(42)

# 1. Importar el mismo Excel que usamos en el script del diccionario.
ruta_excel <- "muestra_estratificada-simulado-de-ataques.xlsx"
base <- read_excel(path = ruta_excel, sheet = "muestra")
info <- read_excel(path = ruta_excel, sheet = "diccionario")
info <- info[match(names(base), info$variable), ]
stopifnot(!anyNA(info$variable), !anyDuplicated(info$variable))
base_original <- base

diccionario <- data.frame(
  nombre = info$variable,
  descripcion = info$descripcion,
  unidad = info$unidad,
  tipo = ifelse(info$tipo %in% c("continua", "conteo"), "numérica", "categórica")
)
print(dim(base))
print(table(diccionario$tipo))
if (interactive()) View(diccionario)

# Guardar solo las tablas necesarias, con nombres fáciles de identificar.
dir.create("resultados/tablas", recursive = TRUE, showWarnings = FALSE)
dir.create("resultados/figuras", recursive = TRUE, showWarnings = FALSE)
guardar_tabla <- function(tabla, nombre) {
  write.csv(tabla, paste0("resultados/tablas/", nombre, ".csv"),
            row.names = FALSE, fileEncoding = "UTF-8", na = "")
}
guardar_tabla(diccionario, "diccionario_variables")

# 2. Elegir las 12 log_ acordadas para ACP, AF y clustering.
variables_num <- c(
  "log_dur", "log_spkts", "log_sbytes", "log_dbytes", "log_sinpkt", "log_sjit",
  "log_synack", "log_ct_srv_src", "log_ct_srv_dst", "log_ct_src_ltm",
  "log_ct_src_dport_ltm", "log_ct_dst_sport_ltm"
)
X_num <- as.data.frame(base[variables_num])
stopifnot(all(is.finite(as.matrix(X_num))))

# 3. Revisar faltantes, duplicados y valores incompatibles con su definición.
faltantes <- data.frame(variable = names(base), NA_encontrados = colSums(is.na(base)))
faltantes$textos_vacios <- sapply(base, function(x) sum(trimws(as.character(x)) == "", na.rm = TRUE))
columnas_perfil <- setdiff(names(base)[1:45], "id")
columnas_numericas <- names(base)[sapply(base, is.numeric)]
conteos <- info$variable[info$tipo == "conteo"]
valores <- as.matrix(base[columnas_numericas])

calidad <- data.frame(
  control = c("Valores NA", "Textos vacíos", "Filas duplicadas", "ID repetidos",
    "Perfiles originales duplicados sin id", "Celdas negativas", "Celdas no finitas sin NA",
    "Conteos no enteros", "TTL fuera de 0 a 255", "Ventana TCP fuera de 0 a 65535",
    "dur = 0 con spkts > 1 (revisar precisión)", "sbytes < spkts",
```

```r
    "Indicadores usa_http/is_sm_ips_ports fuera de 0/1", "is_ftp_login fuera de 0/1",
    "label fuera de normal/ataque", "label y attack_cat inconsistentes"),
  cantidad = c(sum(is.na(base)), sum(faltantes$textos_vacios), sum(duplicated(base)),
    sum(duplicated(base$id)), sum(duplicated(base[columnas_perfil])),
    sum(valores < 0, na.rm = TRUE), sum(!is.finite(valores) & !is.na(valores)),
    sum(sapply(base[conteos], function(x) sum(abs(x - round(x)) > 1e-8, na.rm = TRUE))),
    sum(base$sttl < 0 | base$sttl > 255 | base$dttl < 0 | base$dttl > 255, na.rm = TRUE),
    sum(base$swin < 0 | base$swin > 65535 | base$dwin < 0 | base$dwin > 65535, na.rm = TRUE),
    sum(base$dur == 0 & base$spkts > 1), sum(base$sbytes < base$spkts),
    sum(!base$usa_http %in% c(0, 1)) + sum(!base$is_sm_ips_ports %in% c(0, 1)),
    sum(!base$is_ftp_login %in% c(0, 1)), sum(!base$label %in% c("normal", "ataque")),
    sum((base$attack_cat == "Normal") != (base$label == "normal")))
)
ftp_revision <- base[!base$is_ftp_login %in% c(0, 1), c("id", "is_ftp_login", "ct_ftp_cmd")]
guardar_tabla(faltantes, "faltantes")
guardar_tabla(calidad, "calidad")
guardar_tabla(ftp_revision, "ftp_revision")
print(calidad)
print(ftp_revision)

# Atípicos: límites de Tukey sobre las 12 log_, sin borrar observaciones.
# fivenum usa bisagras de Tukey: pueden diferir de quantile cuando hay empates.
atipicos <- matrix(FALSE, nrow = nrow(base), ncol = length(variables_num))
limites <- data.frame()
for (i in seq_along(variables_num)) {
  variable <- variables_num[i]
  x <- X_num[[variable]]
  cinco <- fivenum(x)
  q1 <- cinco[2]
  q3 <- cinco[4]
  ric <- q3 - q1
  inferior <- q1 - 1.5 * ric
  superior <- q3 + 1.5 * ric
  atipicos[, i] <- x < inferior | x > superior
  limites <- rbind(limites, data.frame(variable = variable, q1_tukey = q1,
    mediana = cinco[3], q3_tukey = q3, limite_inferior = inferior, limite_superior = superior,
    bigote_inferior = min(x[x >= inferior & x <= superior]),
    bigote_superior = max(x[x >= inferior & x <= superior]), n_atipicos = sum(atipicos[, i])))
}
revision_atipicos <- data.frame(id = base$id, n_atipicos_original = base$n_atipicos,
                                n_atipicos_revision_12 = rowSums(atipicos))
guardar_tabla(limites, "atipicos_limites")
guardar_tabla(revision_atipicos, "atipicos_por_flujo")
cat("Flujos con atípicos en las 12 variables:", sum(rowSums(atipicos) > 0), "\n")

# Registrar qué se hace con cada problema; conservar la base recibida.
decisiones <- data.frame(
  tema = c("Faltantes", "Duplicados", "Valores incompatibles", "Código FTP",
           "Ceros y servicio ninguno", "Atípicos", "Indicador anterior"),
  decision = c("No imputar: no se encontraron faltantes en esta base.",
    "No eliminar filas: no se encontraron duplicados en esta base.",
    "Registrar los controles y revisar incidencias antes de cualquier corrección.",
    "Conservar el código 4 y el flujo; is_ftp_login no entra en X_num ni X_cat.",
    "Conservar: pueden ser valores estructurales, no faltantes.",
    "Conservar todos los flujos; un extremo no demuestra error ni ataque.",
    "No sobrescribir n_atipicos; guardar la revisión de las 12 variables por separado.")
)
guardar_tabla(decisiones, "decisiones_calidad")

# 4. Descriptiva de las 47 numéricas; se incluyen dpkts y las versiones log_.
numericas <- diccionario$nombre[diccionario$tipo == "numérica"]
resumen <- data.frame()
for (variable in numericas) {
  x <- base[[variable]]
  q <- quantile(x, c(0, .25, .5, .75, 1), na.rm = TRUE, names = FALSE, type = 7)
  resumen <- rbind(resumen, data.frame(variable = variable, media = mean(x, na.rm = TRUE),
    desvio = sd(x, na.rm = TRUE), minimo = q[1], q1 = q[2], mediana = q[3], q3 = q[4],
    maximo = q[5], ceros = sum(x == 0, na.rm = TRUE)))
}
guardar_tabla(resumen, "resumen_numerico")
print(resumen)

categoricas <- diccionario$nombre[diccionario$tipo == "categórica"]
```

```r
frecuencias <- data.frame()
for (variable in categoricas) {
  tabla <- table(base[[variable]], useNA = "ifany")
  frecuencias <- rbind(frecuencias, data.frame(variable = variable, categoria = names(tabla),
    frecuencia = as.integer(tabla), porcentaje = 100 * as.integer(tabla) / nrow(base)))
}
guardar_tabla(frecuencias, "frecuencias_categoricas")

# 5. Piloto con psych: Pearson, Bartlett, KMO y MSA por variable.
correlaciones <- cor(X_num, method = "pearson")
kmo <- KMO(correlaciones)
bartlett <- cortest.bartlett(correlaciones, n = nrow(X_num))
msa <- data.frame(variable = names(kmo$MSAi), msa = as.numeric(kmo$MSAi))
piloto <- data.frame(determinante = det(correlaciones), kmo_global = kmo$MSA,
  msa_minima = min(kmo$MSAi), bartlett_chi2 = bartlett$chisq, bartlett_gl = bartlett$df,
  bartlett_p_calculado = bartlett$p.value,
  bartlett_p_informe = ifelse(bartlett$p.value < .001, "< 0,001", as.character(bartlett$p.value)))
guardar_tabla(data.frame(variable = rownames(correlaciones), correlaciones), "correlaciones")
guardar_tabla(msa, "msa_por_variable")
guardar_tabla(piloto, "piloto_supuestos")
print(piloto)
print(msa)
if (kmo$MSA < .6 || min(kmo$MSAi) < .5 || det(correlaciones) <= 1e-5)
  warning("Revisar el piloto; no se cambia X_num automáticamente.")

# 6. Congelar X_cat: cuatro categorías existentes y tres discretizaciones.
# Los terciles no usan la etiqueta de ataque para elegir los cortes.
cortes_dur <- unname(quantile(base$dur, c(1/3, 2/3), type = 7))
cortes_bytes <- unname(quantile(base$sbytes, c(1/3, 2/3), type = 7))
X_cat <- as.data.frame(base[c("tipo_trafico", "proto_g", "service_g", "state_g")])
X_cat$respuesta_destino <- ifelse(base$dpkts == 0, "sin_respuesta", "con_respuesta")
X_cat$duracion_banda <- cut(base$dur, c(-Inf, cortes_dur, Inf), labels = c("corta", "media", "larga"))
X_cat$bytes_origen_banda <- cut(base$sbytes, c(-Inf, cortes_bytes, Inf), labels = c("bajos", "medios", "
    ↪ altos"))
X_cat[] <- lapply(X_cat, factor)
frecuencias_cat <- data.frame()
for (variable in names(X_cat)) {
  tabla <- table(X_cat[[variable]])
  frecuencias_cat <- rbind(frecuencias_cat, data.frame(variable = variable,
    categoria = names(tabla), frecuencia = as.integer(tabla), porcentaje = 100 * as.integer(tabla) / nrow(
    ↪ base)))
}
stopifnot(!anyNA(X_cat), all(frecuencias_cat$porcentaje >= 5))
guardar_tabla(frecuencias_cat, "frecuencias_X_cat")
guardar_tabla(data.frame(variable = c("dur", "sbytes"), corte_1 = c(cortes_dur[1], cortes_bytes[1]),
  corte_2 = c(cortes_dur[2], cortes_bytes[2])), "cortes_discretizacion")
guardar_tabla(data.frame(conjunto = c(rep("X_num", ncol(X_num)), rep("X_cat", ncol(X_cat))),
  variable = c(names(X_num), names(X_cat))), "conjuntos_congelados")

# 7. Guardar la base común y los conjuntos sin crear una carpeta datos ni archivos md.
stopifnot(identical(base, base_original))
saveRDS(base, "resultados/base_limpia.rds")
saveRDS(list(X_num = X_num, X_cat = X_cat, id = base$id, semilla = 42,
  cortes_dur = cortes_dur, cortes_bytes = cortes_bytes,
  estado_X_cat = "Congelado para los análisis de la Parte 2"), "resultados/conjuntos_congelados.rds")
cat("Base y conjuntos guardados en resultados. Excel original conservado.\n")
cat("Gráficos: ejecutar en Python el archivo python/graficos_etapa0.py.\n")
```

`R/01_acp.R` · Etapa 1: análisis de componentes principales

```r
# ETAPA 1. ACP sobre las 12 log_, desde la carpeta principal del proyecto.
set.seed(42)
base <- readRDS("resultados/base_limpia.rds")
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_num <- conjuntos$X_num
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")

# Estandarizar evita que las mayores varianzas determinen las componentes.
acp <- prcomp(X_num, scale. = TRUE)
acp_sin_escalar <- prcomp(X_num, scale. = FALSE)
```

```r
autovalores <- acp$sdev^2
tabla_autovalores <- data.frame(componente = seq_along(autovalores), autovalor = autovalores,
  porcentaje = 100 * autovalores / sum(autovalores),
  acumulado = 100 * cumsum(autovalores) / sum(autovalores))
guardar(tabla_autovalores, "acp_autovalores")
guardar(data.frame(variable = names(X_num), varianza = sapply(X_num, var)), "acp_varianzas")
guardar(data.frame(version = c("estandarizado", "sin_estandarizar"),
  porcentaje_CP1 = c(100 * autovalores[1] / sum(autovalores),
    100 * acp_sin_escalar$sdev[1]^2 / sum(acp_sin_escalar$sdev^2))), "acp_comparacion_escala")

# Kaiser sugiere tres; se contrasta el resultado con el codo y el significado de los ejes.
n_componentes <- sum(autovalores > 1)
correlaciones <- cor(X_num, acp$x)
contribuciones <- 100 * acp$rotation^2
cos2 <- correlaciones^2
guardar(data.frame(variable = rownames(correlaciones), correlaciones[, 1:n_componentes]), "acp_
    ↪ correlaciones")
guardar(data.frame(variable = rownames(contribuciones), contribuciones[, 1:n_componentes]), "acp_
    ↪ contribuciones")
guardar(data.frame(variable = rownames(cos2), cos2[, 1:n_componentes]), "acp_cos2")
puntajes <- acp$x[, 1:n_componentes, drop = FALSE]
guardar(data.frame(id = base$id, tipo_trafico = base$tipo_trafico, puntajes), "acp_puntajes")

# Distancia en el espacio retenido, ajustada por la varianza de cada componente.
distancia <- rowSums(sweep(puntajes, 2, acp$sdev[1:n_componentes], "/")^2)
extremos <- order(distancia, decreasing = TRUE)[1:3]
guardar(data.frame(base[extremos, c("id", "tipo_trafico", "dur", "spkts", "dpkts", "sbytes", "dbytes")],
  puntajes[extremos, , drop = FALSE], distancia = distancia[extremos]), "acp_extremos")
saveRDS(list(modelo = acp, n_componentes = n_componentes, puntajes = puntajes), "resultados/acp.rds")
print(tabla_autovalores)
cat("Componentes retenidas:", n_componentes, "\n")
```

`R/02_af.R` · Etapa 2: análisis factorial

```r
# ETAPA 2. Análisis factorial exploratorio y revisión de soluciones impropias.
library(psych)
set.seed(42)
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_num <- conjuntos$X_num
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")
R <- cor(X_num)
kmo <- KMO(R)
bartlett <- cortest.bartlett(R, n = nrow(X_num))
stopifnot(det(R) > .00001, kmo$MSA >= .6, min(kmo$MSAi) >= .5, bartlett$p.value < .05)

# El análisis paralelo compara los autovalores observados con datos aleatorios.
# No se dibuja en R: los resultados se exportan para Matplotlib.
options(mc.cores = 1)
paralelo <- fa.parallel(X_num, fa = "both", fm = "minres", n.iter = 100,
  quant = .95, plot = FALSE)
guardar(data.frame(dimension = seq_along(paralelo$fa.values),
  factor_observado = paralelo$fa.values, factor_simulado = paralelo$fa.sim,
  componente_observado = paralelo$pc.values, componente_simulado = paralelo$pc.sim), "af_paralelo")

# Se retienen los factores sugeridos por el análisis paralelo y se revisa su interpretación.
n_factores <- paralelo$nfact
minres <- fa(X_num, nfactors = n_factores, fm = "minres", rotate = "varimax", scores = "regression")
guardar(data.frame(metodo = "minres", n_factores = n_factores,
  comunalidad_max = max(minres$communality), unicidad_min = min(minres$uniquenesses),
  heywood = any(minres$uniquenesses < 0)), "af_diagnostico_minres")

# Minres produce una unicidad negativa: no se oculta ni se elimina una variable acordada.
# ML con límite positivo permite una solución descriptiva; se informa el parámetro en el límite.
af <- factanal(X_num, factors = n_factores, rotation = "varimax", scores = "regression",
  control = list(lower = .005))
cargas <- unclass(af$loadings)
colnames(cargas) <- paste0("F", seq_len(n_factores))
patron <- cargas
patron[abs(patron) < .4] <- NA
diferencia <- apply(abs(cargas), 1, function(x) { x <- sort(x, decreasing = TRUE); x[1] - x[2] })
```

```r
n_cargas <- rowSums(abs(cargas) >= .4)
guardar(data.frame(variable = rownames(cargas), cargas, comunalidad = 1 - af$uniquenesses,
  unicidad = af$uniquenesses, diferencia = diferencia, cargas_mayores_040 = n_cargas), "af_cargas_completas")
guardar(data.frame(variable = rownames(cargas), patron, comunalidad = 1 - af$uniquenesses,
  unicidad = af$uniquenesses), "af_patron")
guardar(data.frame(factor = colnames(cargas), suma_cargas2 = colSums(cargas^2),
  porcentaje_total = 100 * colSums(cargas^2) / ncol(X_num),
  porcentaje_comun = 100 * colSums(cargas^2) / sum(cargas^2)), "af_varianza")
residuos <- R - tcrossprod(cargas)
rmsr <- sqrt(mean(residuos[lower.tri(residuos)]^2))
guardar(data.frame(n_factores = n_factores, rmsr = rmsr,
  chi2 = unname(af$STATISTIC), gl = af$dof, p_modelo = af$PVAL,
  comunalidad_min = min(1 - af$uniquenesses), unicidad_min = min(af$uniquenesses),
  unicidades_limite = sum(af$uniquenesses <= .00501),
  cruces_ambiguos = sum(n_cargas > 1 & diferencia < .2)), "af_ajuste")
guardar(data.frame(id = conjuntos$id, af$scores), "af_puntajes")
saveRDS(list(modelo = af, n_factores = n_factores, cargas = cargas, paralelo = paralelo), "resultados/af.rds")
print(af$loadings, cutoff = .4)
print(af$uniquenesses)
```

`R/aux_diagnostico_af.R` · Etapa 2 (auxiliar): diagnóstico del caso Heywood

```r
# AUXILIAR (no forma parte de ejecutar_todo.R). Diagnóstico del caso Heywood en el AF.
# Ejecutar desde la raíz del repo:  Rscript R/aux_diagnostico_af.R
# Compara 3 vs 4 factores x (minres, ml, pa) x (varimax, oblimin) y guarda
# resultados/tablas/af_diagnostico_comparativo.csv y af_diagnostico_autovalores.csv.
# Sirve de evidencia para justificar cuántos factores retiene R/02_af.R.
library(psych)
set.seed(42)
X <- readRDS("resultados/conjuntos_congelados.rds")$X_num
R <- cor(X)
p <- ncol(X)

# Bloques esperados por H1 (docs/etapa0_pregunta_hipotesis.md); solo para chequear interpretabilidad.
bloque_h1 <- c(log_spkts = "vol", log_sbytes = "vol", log_dbytes = "vol",
  log_dur = "tmp", log_sinpkt = "tmp", log_sjit = "tmp", log_synack = "tmp",
  log_ct_srv_src = "ctx", log_ct_srv_dst = "ctx", log_ct_src_ltm = "ctx",
  log_ct_src_dport_ltm = "ctx", log_ct_dst_sport_ltm = "ctx")

# 1. Autovalores: ¿es sustantivo el 4.º factor?
# Matriz reducida = R con la diagonal reemplazada por las comunalidades iniciales (SMC).
smc_ <- smc(R)
Rred <- R; diag(Rred) <- smc_
av_red <- eigen(Rred, symmetric = TRUE, only.values = TRUE)$values
av_cor <- eigen(R, symmetric = TRUE, only.values = TRUE)$values
set.seed(42)
pa <- suppressWarnings(fa.parallel(X, fa = "both", fm = "minres", n.iter = 100,
  quant = .95, plot = FALSE))
# Con n = 5000 el análisis paralelo detecta casi cualquier cosa por encima del azar:
# se agrega la varianza que aporta cada autovalor (autovalor reducido / suma de positivos).
aut <- data.frame(factor = 1:p, autovalor_R = av_cor, autovalor_reducida_smc = av_red,
  pct_var_total = 100 * av_cor / p,
  pct_var_comun = 100 * pmax(av_red, 0) / sum(pmax(av_red, 0)),
  fa_observado = pa$fa.values, fa_simulado = pa$fa.sim)
write.csv(aut, "resultados/tablas/af_diagnostico_autovalores.csv", row.names = FALSE, fileEncoding = "UTF-8")
cat("\n== Autovalores (los 6 primeros) ==\n"); print(round(aut[1:6, ], 3), row.names = FALSE)
cat("Sugerencia del análisis paralelo (minres):", pa$nfact, "factores.\n")
cat("Kaiser sobre la matriz reducida (autovalor > 1):", sum(av_red > 1), "factores.\n")
cat("Autovalores reducidos > 0:", sum(av_red > 0), "\n")

# 2. Comparativo de soluciones
filas <- list()
for (k in 3:4) for (m in c("minres", "ml", "pa")) for (rot in c("varimax", "oblimin")) {
  aviso <- character()
  f <- withCallingHandlers(
    tryCatch(fa(X, nfactors = k, fm = m, rotate = rot, n.obs = nrow(X)), error = function(e) NULL),
    warning = function(w) { aviso <<- c(aviso, conditionMessage(w)); invokeRestart("muffleWarning") })
  if (is.null(f)) next
```

```r
  L <- unclass(f$loadings)
  com <- f$communality
  uni <- f$uniquenesses
  ord <- t(apply(abs(L), 1, sort, decreasing = TRUE))
  dif <- ord[, 1] - ord[, 2]
  n04 <- rowSums(abs(L) >= .4)
  # Interpretabilidad: cada variable al factor de mayor carga; ¿cada factor agrupa un solo bloque H1?
  asig <- apply(abs(L), 1, which.max)
  puros <- sum(sapply(split(bloque_h1[rownames(L)], asig), function(b) length(unique(b)) == 1))
  prop <- f$Vaccounted
  filas[[length(filas) + 1]] <- data.frame(
    n_factores = k, metodo = m, rotacion = rot,
    heywood = any(com >= 1 | uni <= .005),
    comunalidad_max = max(com), unicidad_min = min(uni),
    rmsr = f$rms, tli = f$TLI, rmsea = f$RMSEA[1],
    pct_var_acum = 100 * tail(prop["Cumulative Var", ], 1),
    sin_carga_040 = sum(n04 == 0),
    cruces_dif_menor_020 = sum(n04 > 1 & dif < .2),
    factores_con_un_solo_bloque_H1 = puros,
    avisos = length(aviso))
}
comp <- do.call(rbind, filas)
write.csv(comp, "resultados/tablas/af_diagnostico_comparativo.csv", row.names = FALSE, fileEncoding = "UTF
    ↪ -8")
cat("\n== Comparativo (varimax y oblimin) ==\n")
print(format(comp, digits = 3), row.names = FALSE)

# 3. Estructura de la solución de 3 y 4 factores con minres/varimax (la del análisis paralelo)
for (k in 3:4) {
  f <- suppressWarnings(fa(X, nfactors = k, fm = "minres", rotate = "varimax"))
  cat("\n== minres + varimax,", k, "factores (cargas < 0,40 ocultas) ==\n")
  print(f$loadings, cutoff = .4, digits = 2)
  cat("Comunalidades:\n"); print(round(f$communality, 3))
}
# En 4 factores, ¿cuánto aporta el 4.º y qué hace? Correlación entre variables ct_* de cada puerto/servicio:
cat("\nCorrelaciones dentro del bloque ct_*:\n"); print(round(R[grep("ct_", rownames(R)), grep("ct_",
    ↪ colnames(R))], 2))
cat("\nCorrelación log_spkts con log_sbytes:", round(R["log_spkts", "log_sbytes"], 3),
    "| con log_dbytes:", round(R["log_spkts", "log_dbytes"], 3), "\n")
```

R/03_cluster.R · Etapa 3: conglomerados de observaciones y de variables

```r
# ETAPA 3. Ward y k-means sobre las 12 log_ estandarizadas.
library(cluster)
set.seed(42)
base <- readRDS("resultados/base_limpia.rds")
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_num <- conjuntos$X_num
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")
Z <- scale(X_num)
distancias <- dist(Z, method = "euclidean")
jerarquico <- hclust(distancias, method = "ward.D2")

# Codo: suma de cuadrados dentro de grupos. Silhouette sobre los 5.000 flujos.
metricas <- data.frame()
modelos_km <- list()
for (k in 2:8) {
  set.seed(42)
  km <- kmeans(Z, centers = k, nstart = 25, iter.max = 100)
  ward <- cutree(jerarquico, k)
  wss_ward <- 0
  for (g in 1:k) {
    grupo <- Z[ward == g, , drop = FALSE]
    wss_ward <- wss_ward + sum(sweep(grupo, 2, colMeans(grupo), "-")^2)
  }
  metricas <- rbind(metricas, data.frame(k = k, wss_ward = wss_ward, wss_kmeans = km$tot.withinss,
    silhouette_ward = mean(silhouette(ward, distancias)[, 3]),
    silhouette_kmeans = mean(silhouette(km$cluster, distancias)[, 3])))
  modelos_km[[as.character(k)]] <- km
  cat("Evaluado k =", k, "\n")
```

```r
}
guardar(metricas, "cluster_metricas")

# Mayor silhouette de k-means entre los candidatos; el informe contrasta con el codo y Ward.
k_elegido <- metricas$k[which.max(metricas$silhouette_kmeans)]
km <- modelos_km[[as.character(k_elegido)]]
ward <- cutree(jerarquico, k_elegido)
guardar(as.data.frame.matrix(table(Ward = ward, Kmeans = km$cluster)), "cluster_cruce_ward_kmeans")

# Comparación de particiones sin depender del número arbitrario de las etiquetas.
ari <- function(a, b) {
  t <- table(a, b)
  observado <- sum(choose(t, 2))
  esperado <- sum(choose(rowSums(t), 2)) * sum(choose(colSums(t), 2)) / choose(sum(t), 2)
  maximo <- (sum(choose(rowSums(t), 2)) + sum(choose(colSums(t), 2))) / 2
  (observado - esperado) / (maximo - esperado)
}

# Los puntajes de ACP conservan la ponderación por sus autovalores: no se reescalan.
acp <- readRDS("resultados/acp.rds")
set.seed(42)
km_acp <- kmeans(acp$puntajes, centers = k_elegido, nstart = 25, iter.max = 100)
ward_acp <- cutree(hclust(dist(acp$puntajes), method = "ward.D2"), k_elegido)
guardar(as.data.frame.matrix(table(Kmeans_log = km$cluster, Kmeans_ACP = km_acp$cluster)), "cluster_cruce_
    ↪ acp")
guardar(as.data.frame.matrix(table(Ward_log = ward, Ward_ACP = ward_acp)), "cluster_cruce_ward_acp")
guardar(data.frame(k = k_elegido, ARI_ward_kmeans = ari(ward, km$cluster),
  ARI_kmeans_acp = ari(km$cluster, km_acp$cluster), ARI_ward_acp = ari(ward, ward_acp)), "cluster_
    ↪ comparaciones")

originales <- sub("^log_", "", names(X_num))
guardar(aggregate(base[originales], list(cluster = km$cluster), mean), "cluster_medias_originales")
guardar(aggregate(as.data.frame(Z), list(cluster = km$cluster), mean), "cluster_medias_estandarizadas")
guardar(data.frame(cluster = 1:k_elegido, n = as.integer(table(km$cluster)),
  porcentaje = 100 * as.integer(table(km$cluster)) / nrow(base)), "cluster_tamanos")
guardar(as.data.frame(table(cluster = km$cluster, tipo_trafico = base$tipo_trafico)), "cluster_contexto")
guardar(aggregate(as.numeric(base$label == "ataque"), list(cluster = km$cluster), mean), "cluster_
    ↪ proporcion_ataques")

# Cluster de variables: distancia euclidiana entre sus columnas estandarizadas, Ward.D2.
jerarquico_vars <- hclust(dist(t(Z)), method = "ward.D2")
guardar(data.frame(variable = names(X_num), bloque = cutree(jerarquico_vars, 3)), "cluster_variables_
    ↪ bloques")
guardar(data.frame(izquierda = jerarquico$merge[, 1], derecha = jerarquico$merge[, 2],
  altura = jerarquico$height), "cluster_dendrograma")
guardar(data.frame(izquierda = jerarquico_vars$merge[, 1], derecha = jerarquico_vars$merge[, 2],
  altura = jerarquico_vars$height), "cluster_dendrograma_variables")
base$cluster <- factor(km$cluster)
base$cluster_ward <- factor(ward)
base$cluster_acp <- factor(km_acp$cluster)
saveRDS(base, "resultados/base_con_clusters.rds")
guardar(data.frame(id = base$id, cluster = km$cluster, cluster_ward = ward,
  cluster_acp = km_acp$cluster), "cluster_asignaciones")
saveRDS(list(k = k_elegido, kmeans = km, jerarquico = jerarquico,
  jerarquico_variables = jerarquico_vars), "resultados/clustering.rds")
print(metricas)
cat("k elegido:", k_elegido, "\n")
```

`R/04_acm.R` · Etapa 4: correspondencias simple y múltiples

```r
# ETAPA 4. ACS, ACM y proyección suplementaria del cluster.
library(ca)
library(FactoMineR)
set.seed(42)
base <- readRDS("resultados/base_con_clusters.rds")
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_cat <- conjuntos$X_cat
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")

# Las siete variables activas ya incluyen tres numéricas discretizadas en Etapa 0.
```

```r
# Sus modalidades superan 5%; no se fusionan categorías adicionales.
frecuencias <- read.csv("resultados/tablas/frecuencias_X_cat.csv")
stopifnot(all(frecuencias$porcentaje >= 5), all(sapply(X_cat, is.factor)))

# ACS: tipo de tráfico y estado agrupado, para estudiar la tercera hipótesis.
tabla <- table(X_cat$tipo_trafico, X_cat$state_g)
acs <- ca::ca(tabla, nd = 2)
chi <- chisq.test(tabla, correct = FALSE)
guardar(data.frame(tipo_trafico = rownames(tabla), unclass(tabla)), "acs_contingencia")
guardar(data.frame(tipo_trafico = rownames(tabla), unclass(100 * prop.table(tabla, 1))), "acs_porcentajes_
    ↪ fila")
guardar(data.frame(dimension = seq_along(acs$sv), autovalor = acs$sv^2,
  porcentaje = 100 * acs$sv^2 / sum(acs$sv^2)), "acs_inercia")
guardar(data.frame(chi2 = unname(chi$statistic), gl = unname(chi$parameter), p = chi$p.value,
  cramer_v = sqrt(unname(chi$statistic) / (sum(tabla) * min(nrow(tabla) - 1, ncol(tabla) - 1))),
  frecuencia_esperada_min = min(chi$expected)), "acs_asociacion")
guardar(data.frame(categoria = rownames(tabla),
  sweep(acs$rowcoord[, 1:2, drop = FALSE], 2, acs$sv[1:2], "*")), "acs_filas")
guardar(data.frame(categoria = colnames(tabla),
  sweep(acs$colcoord[, 1:2, drop = FALSE], 2, acs$sv[1:2], "*")), "acs_columnas")

# Cluster es suplementario: no interviene en la construcción de los ejes.
datos_acm <- data.frame(X_cat, cluster = base$cluster)
Q <- ncol(X_cat)
J <- sum(sapply(X_cat, nlevels))
acm <- FactoMineR::MCA(datos_acm, quali.sup = ncol(datos_acm), ncp = J - Q, graph = FALSE)
ajustado <- ca::mjca(X_cat, nd = NA, lambda = "adjusted")
indicador <- ca::mjca(X_cat, nd = NA, lambda = "indicator")
# ca 0.72 entrega valores singulares: al cuadrarlos se obtienen los autovalores.
stopifnot(max(abs(indicador$sv^2 - acm$eig[, 1])) < 1e-8)
autovalores <- data.frame(dimension = seq_len(nrow(acm$eig)), autovalor = acm$eig[, 1],
  porcentaje_crudo = acm$eig[, 2], acumulado_crudo = acm$eig[, 3],
  autovalor_ajustado = 0, porcentaje_ajustado = 0)
n_aj <- length(ajustado$sv)
autovalores$autovalor_ajustado[1:n_aj] <- ajustado$sv^2
autovalores$porcentaje_ajustado[1:n_aj] <- 100 * ajustado$inertia.e
autovalores$acumulado_ajustado <- cumsum(autovalores$porcentaje_ajustado)
guardar(autovalores, "acm_autovalores")
guardar(data.frame(Q = Q, J = J, inercia_cruda = J / Q - 1,
  umbral = 1 / Q, ejes_sobre_umbral = sum(acm$eig[, 1] > 1 / Q),
  inercia_total_ajustada = ajustado$inertia.t,
  porcentaje_plano_ajustado = sum(autovalores$porcentaje_ajustado[1:2])), "acm_resumen")

# Se conservan todos los resultados; el informe interpreta el plano de dos dimensiones.
categorias <- ajustado$levelnames
coord <- data.frame(categoria = categorias, acm$var$coord)
contrib <- data.frame(categoria = categorias, acm$var$contrib)
cos2 <- data.frame(categoria = categorias, acm$var$cos2)
guardar(coord, "acm_categorias_coordenadas")
guardar(contrib, "acm_categorias_contribuciones")
guardar(cos2, "acm_categorias_cos2")
guardar(data.frame(variable = rownames(acm$var$eta2), acm$var$eta2), "acm_eta2")
guardar(data.frame(cluster = levels(base$cluster), acm$quali.sup$coord), "acm_cluster_suplementario")
guardar(data.frame(cluster = levels(base$cluster), acm$quali.sup$cos2), "acm_cluster_cos2")
guardar(data.frame(id = base$id, cluster = base$cluster, acm$ind$coord[, 1:2]), "acm_individuos")

# Cruces para interpretar tres asociaciones con frecuencias observadas, sin causalidad.
for (variable in c("respuesta_destino", "duracion_banda", "bytes_origen_banda")) {
  t <- table(X_cat$tipo_trafico, X_cat[[variable]])
  guardar(data.frame(tipo_trafico = rownames(t), unclass(100 * prop.table(t, 1))), paste0("acm_cruce_",
    ↪ variable))
}
saveRDS(list(acs = acs, acm = acm, ajustado = ajustado, dimensiones_interpretadas = 2), "resultados/
    ↪ correspondencias.rds")
print(autovalores)
print(acm$var$eta2[, 1:2])
```

`R/05_integracion.R` · Etapa 6: integración de técnicas

```r
# ETAPA 6. Síntesis de resultados de las técnicas multivariadas.
set.seed(42)
```

```r
base <- readRDS("resultados/base_con_clusters.rds")
X <- readRDS("resultados/conjuntos_congelados.rds")$X_num
acp <- readRDS("resultados/acp.rds")
af <- readRDS("resultados/af.rds")
km <- readRDS("resultados/clustering.rds")
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")

# Evidencia de la hipótesis de asociaciones, con tamaños del efecto.
asociaciones <- data.frame()
for (variable in c("proto_g", "service_g", "state_g")) {
  t <- table(base$tipo_trafico, base[[variable]])
  prueba <- chisq.test(t, correct = FALSE)
  asociaciones <- rbind(asociaciones, data.frame(variable = variable,
    chi2 = unname(prueba$statistic), gl = unname(prueba$parameter), p = prueba$p.value,
    cramer_v = sqrt(unname(prueba$statistic) / (sum(t) * min(dim(t) - 1)))))
}
guardar(asociaciones, "integracion_asociaciones")

integracion <- data.frame(
  tecnica = c("ACP", "AF", "Clustering", "ACS", "ACM"),
  estructura = c(paste(acp$n_componentes, "componentes; 74,51% de varianza"),
    paste(af$n_factores, "factores; solución ML con unicidad en el límite"),
    paste("Tres bloques de variables y", km$k, "grupos de observaciones"),
    "Tipo de tráfico por estado; V de Cramér = 0,405",
    "Plano de dos dimensiones; 87,40% de inercia ajustada"),
  interpretacion = c("Ejes mixtos: complejidad frente a repetición; volumen y actividad",
    "Tiempo, volumen, repetición por puertos y repetición por servicio",
    "AF reconoce los bloques; ARI Ward/KM = 0,813 y silhouette = 0,333; las CP cambian fronteras",
    "Normal-CON_otros y Exploits-FIN coinciden con asociaciones del ACM",
    "Respuesta/protocolo/estado y oposición entre estados y tipos de servicio")
)
guardar(integracion, "integracion_tecnicas")
print(integracion)
```

`R/06_logistica.R` · Regresión logística

```r
# ETAPA 6. Regresión logística: ¿es ataque (1) o normal (0)?
# Ejecutar desde la carpeta principal del proyecto: Rscript R/06_logistica.R
#
# Diseño (decidido por el grupo/orquestador):
#  - Validación: partición estratificada por label, 70 % entrenamiento / 30 % prueba,
#    dentro de la muestra de 5.000 flujos. El testing set original de UNSW-NB15 no está
#    en el repositorio y habría que prepararlo igual que la muestra.
#  - Predictores: los 3 puntajes del ACP + proto_g, service_g, state_g.
#  - Prohibidos: TTL (sttl, dttl, ttl_origen, perfil_estado_ttl, ct_state_ttl: separación
#    perfecta por artefacto del laboratorio), tipo_trafico, attack_cat y n_atipicos
#    (son la respuesta "disfrazada" o derivan de ella).
#  - El ACP se reajusta SOLO con el entrenamiento para que el conjunto de prueba no
#    influya en los puntajes (evita fuga de información).
library(MASS)   # confint() por verosimilitud perfilada para glm
library(pROC)
set.seed(42)

base <- readRDS("resultados/base_limpia.rds")
conjuntos <- readRDS("resultados/conjuntos_congelados.rds")
X_num <- conjuntos$X_num
acp_completo <- readRDS("resultados/acp.rds")
n_comp <- acp_completo$n_componentes   # 3 componentes (Kaiser), como en el informe
dir.create("resultados/tablas", recursive = TRUE, showWarnings = FALSE)
guardar <- function(x, nombre) write.csv(x, paste0("resultados/tablas/log_reg_", nombre, ".csv"),
  row.names = FALSE, fileEncoding = "UTF-8")
coma <- function(x, d = 3) formatC(x, format = "f", digits = d)

# 1. Respuesta: ataque = 1, normal = 0 (en la base label es texto "ataque"/"normal").
stopifnot(all(base$label %in% c("ataque", "normal")))
base$y <- as.integer(base$label == "ataque")

# 2. Partición estratificada 70/30 por label: se sortea dentro de cada clase para que
# ambos conjuntos conserven ~51,8 % de ataques.
idx_train <- unlist(lapply(split(seq_len(nrow(base)), base$y), function(i)
```

```r
  sample(i, size = round(0.7 * length(i)))))
idx_train <- sort(idx_train)
idx_test <- setdiff(seq_len(nrow(base)), idx_train)
stopifnot(length(intersect(idx_train, idx_test)) == 0)
particion <- data.frame(conjunto = c("entrenamiento", "prueba"),
  n = c(length(idx_train), length(idx_test)),
  n_ataques = c(sum(base$y[idx_train]), sum(base$y[idx_test])),
  pct_ataques = c(100 * mean(base$y[idx_train]), 100 * mean(base$y[idx_test])))
guardar(particion, "particion")
guardar(data.frame(id = base$id, conjunto = ifelse(seq_len(nrow(base)) %in% idx_train,
  "entrenamiento", "prueba")), "particion_ids")
print(particion)

# 3. ACP reajustado SOLO con el entrenamiento; la prueba se proyecta con predict(), que
# usa las medias y desvíos del entrenamiento.
acp_tr <- prcomp(X_num[idx_train, ], scale. = TRUE)
# Los signos de una componente son arbitrarios. Se alinean con el ACP del informe (5.000):
# si la carga de entrenamiento apunta al lado contrario, se invierte la componente.
signos <- sapply(1:n_comp, function(k)
  sign(sum(acp_tr$rotation[, k] * acp_completo$modelo$rotation[, k])))
cat("Signo relativo al ACP completo (1 = igual, -1 = invertido):", signos, "\n")
acp_tr$rotation[, 1:n_comp] <- sweep(acp_tr$rotation[, 1:n_comp, drop = FALSE], 2, signos, "*")
# Se recalculan los puntajes de entrenamiento con la rotación ya corregida.
nombres_cp <- paste0("PC", 1:n_comp)
P_tr <- predict(acp_tr, X_num[idx_train, ])[, 1:n_comp]
P_te <- predict(acp_tr, X_num[idx_test, ])[, 1:n_comp]
colnames(P_tr) <- colnames(P_te) <- nombres_cp

# Comparación de cargas: entrenamiento vs. ACP con los 5.000.
dif_cargas <- acp_tr$rotation[, 1:n_comp] - acp_completo$modelo$rotation[, 1:n_comp]
comp_cargas <- data.frame(variable = rownames(acp_tr$rotation),
  setNames(as.data.frame(acp_tr$rotation[, 1:n_comp]), paste0(nombres_cp, "_entrenamiento")),
  setNames(as.data.frame(acp_completo$modelo$rotation[, 1:n_comp]), paste0(nombres_cp, "_completo")))
guardar(comp_cargas, "acp_cargas_comparacion")
resumen_cargas <- data.frame(componente = nombres_cp, signo_invertido = signos == -1,
  max_dif_absoluta = apply(abs(dif_cargas), 2, max),
  correlacion = sapply(1:n_comp, function(k)
    cor(acp_tr$rotation[, k], acp_completo$modelo$rotation[, k])),
  pct_varianza_entrenamiento = 100 * acp_tr$sdev[1:n_comp]^2 / sum(acp_tr$sdev^2))
guardar(resumen_cargas, "acp_cargas_resumen")
print(resumen_cargas)

# 4. Categóricas con categoría de referencia explícita: la más frecuente de cada una,
# así las demás se comparan contra la situación "típica" y el error estándar de la
# referencia es el menor posible: tcp (70,8 %), ninguno (60,0 %), FIN (69,3 %).
ref <- function(x, nivel) relevel(factor(x), ref = nivel)
datos <- data.frame(y = base$y, id = base$id, P = rbind(P_tr, P_te)[order(c(idx_train, idx_test)), ],
  proto_g = ref(base$proto_g, "tcp"), service_g = ref(base$service_g, "ninguno"),
  state_g = ref(base$state_g, "FIN"))
names(datos) <- sub("^P\\.", "", names(datos))
train <- datos[idx_train, ]
test <- datos[idx_test, ]
stopifnot(identical(train$id, base$id[idx_train]))

# 5. Dos modelos anidados. M1: solo los puntajes. M2: M1 + categóricas.
M1 <- glm(y ~ PC1 + PC2 + PC3, data = train, family = binomial)
aviso <- NULL
M2 <- withCallingHandlers(
  glm(y ~ PC1 + PC2 + PC3 + proto_g + service_g + state_g, data = train, family = binomial),
  warning = function(w) { aviso <<- c(aviso, conditionMessage(w)); invokeRestart("muffleWarning") })
if (!is.null(aviso)) cat("AVISO al ajustar M2:", aviso, "\n")

# 6. Comparación: AIC, devianza, test de razón de verosimilitud y AUC en prueba.
auc_prueba <- function(m) auc(roc(test$y, as.numeric(predict(m, test, type = "response")),
  levels = c(0, 1), direction = "<", quiet = TRUE))
mcfadden <- function(m) 1 - m$deviance / m$null.deviance
lr <- anova(M1, M2, test = "Chisq")
comparacion <- data.frame(modelo = c("M1: puntajes ACP", "M2: puntajes ACP + proto + service + state"),
  gl_residuales = c(M1$df.residual, M2$df.residual), parametros = c(length(coef(M1)), length(coef(M2))),
  AIC = c(AIC(M1), AIC(M2)), BIC = c(BIC(M1), BIC(M2)),
  devianza = c(deviance(M1), deviance(M2)),
  LR_chi2 = c(NA, lr$Deviance[2]), LR_gl = c(NA, lr$Df[2]), LR_p = c(NA, lr$`Pr(>Chi)`[2]),
  AUC_prueba = c(as.numeric(auc_prueba(M1)), as.numeric(auc_prueba(M2))),
```

```r
  pseudoR2_McFadden = c(mcfadden(M1), mcfadden(M2)))
guardar(comparacion, "comparacion_modelos")
print(comparacion)

# 7. Elección: M2 si mejora el AIC y el LR es significativo (la ganancia no es azar);
# si no, M1 por parsimonia. La decisión sale de los números, no se fija a mano.
elegido <- if (AIC(M2) < AIC(M1) && lr$`Pr(>Chi)`[2] < 0.05) "M2" else "M1"
modelo <- if (elegido == "M2") M2 else M1
cat("Modelo elegido:", elegido, "\n")

# 8. Coeficientes, OR e IC 95 % por verosimilitud perfilada (más fiable que Wald cuando
# hay categorías raras o coeficientes grandes). Se incluye también el IC de Wald.
sm <- summary(modelo)$coefficients
ic_perfil <- suppressMessages(confint(modelo))
se <- sm[, 2]
coefs <- data.frame(termino = rownames(sm), estimacion = sm[, 1], error_estandar = se,
  z = sm[, 3], p = sm[, 4], OR = exp(sm[, 1]),
  OR_IC95_inf = exp(ic_perfil[, 1]), OR_IC95_sup = exp(ic_perfil[, 2]),
  OR_Wald_IC95_inf = exp(sm[, 1] - 1.96 * se), OR_Wald_IC95_sup = exp(sm[, 1] + 1.96 * se),
  tipo_IC = "verosimilitud perfilada (Wald en columnas aparte)")
guardar(coefs, "coeficientes")
# Los OR de PC1-PC3 son "por cada desvío (1 unidad estandarizada) del puntaje", porque
# el puntaje de cada componente tiene desvío = sdev de la componente: el OR por 1 desvío
# del puntaje es exp(beta * sdev). Se guarda también esa versión, que es la comparable.
sdev_tr <- acp_tr$sdev[1:n_comp]
or_desvio <- data.frame(termino = nombres_cp, desvio_puntaje = sdev_tr,
  OR_por_unidad = exp(coef(modelo)[nombres_cp]),
  OR_por_desvio = exp(coef(modelo)[nombres_cp] * sdev_tr),
  OR_desvio_IC95_inf = exp(ic_perfil[nombres_cp, 1] * sdev_tr),
  OR_desvio_IC95_sup = exp(ic_perfil[nombres_cp, 2] * sdev_tr))
guardar(or_desvio, "or_por_desvio")
print(coefs[, 1:8]); print(or_desvio)

# 9. Desempeño en prueba con umbral 0,5. AUC con IC 95 % de DeLong.
p_te <- as.numeric(predict(modelo, test, type = "response"))
roc_te <- roc(test$y, p_te, levels = c(0, 1), direction = "<", quiet = TRUE)
ic_auc <- ci.auc(roc_te, method = "delong")
guardar(data.frame(AUC = as.numeric(auc(roc_te)), IC95_inf = ic_auc[1], IC95_sup = ic_auc[3],
  metodo_IC = "DeLong", modelo = elegido), "auc")
pred <- as.integer(p_te >= 0.5)
VP <- sum(pred == 1 & test$y == 1); VN <- sum(pred == 0 & test$y == 0)
FP <- sum(pred == 1 & test$y == 0); FN <- sum(pred == 0 & test$y == 1)
guardar(data.frame(real = c("normal", "normal", "ataque", "ataque"),
  predicho = c("normal", "ataque", "normal", "ataque"), n = c(VN, FP, FN, VP)), "confusion")
metricas <- data.frame(umbral = 0.5, exactitud = (VP + VN) / length(pred),
  sensibilidad = VP / (VP + FN), especificidad = VN / (VN + FP),
  precision = VP / (VP + FP), F1 = 2 * VP / (2 * VP + FP + FN))
guardar(metricas, "metricas")
print(metricas); print(ic_auc)

# Curva ROC (puntos para graficar) y probabilidades predichas en prueba.
guardar(data.frame(umbral = roc_te$thresholds, fpr = 1 - roc_te$specificities,
  tpr = roc_te$sensitivities), "roc_curva")
guardar(data.frame(id = test$id, y = test$y, p = p_te), "predicciones_prueba")

# 10. Calibración por deciles de probabilidad predicha (en prueba).
dec <- cut(p_te, breaks = unique(quantile(p_te, seq(0, 1, .1))), include.lowest = TRUE, labels = FALSE)
calib <- do.call(rbind, lapply(sort(unique(dec)), function(d) data.frame(decil = d,
  n = sum(dec == d), p_media = mean(p_te[dec == d]), prop_observada = mean(test$y[dec == d]))))
guardar(calib, "calibracion")
print(calib)

# 11. Multicolinealidad: VIF de cada columna de la matriz de diseño y correlaciones.
# (Las dummies de un mismo factor se correlacionan entre sí por construcción.)
mm <- model.matrix(modelo)[, -1, drop = FALSE]
vif <- sapply(seq_len(ncol(mm)), function(j) 1 / (1 - summary(lm(mm[, j] ~ mm[, -j]))$r.squared))
guardar(data.frame(termino = colnames(mm), VIF = vif), "vif")
guardar(data.frame(termino = colnames(mm), round(cor(mm), 4)), "correlaciones_predictores")
print(data.frame(termino = colnames(mm), VIF = round(vif, 2)))

# 12. Separación: coeficientes enormes, EE gigantes o probabilidades ajustadas en 0/1.
ajustadas <- fitted(modelo)
separacion <- data.frame(
```

```r
  max_abs_coef = max(abs(coef(modelo))), max_error_estandar = max(se),
  ajustadas_menor_1e8 = sum(ajustadas < 1e-8), ajustadas_mayor_1_menos_1e8 = sum(ajustadas > 1 - 1e-8),
  aviso_glm = ifelse(is.null(aviso), "sin avisos", paste(unique(aviso), collapse = " | ")),
  convergio = modelo$converged, iteraciones = modelo$iter)
guardar(separacion, "separacion")
print(separacion)
# Tablas cruzadas categoría x respuesta en entrenamiento: celdas vacías delatarían separación.
cruces <- do.call(rbind, lapply(c("proto_g", "service_g", "state_g"), function(v) {
  t <- table(train[[v]], train$y)
  data.frame(variable = v, categoria = rownames(t), n_normal = t[, "0"], n_ataque = t[, "1"],
    pct_ataque = 100 * t[, "1"] / rowSums(t))
}))
guardar(cruces, "cruces_categoricas")
print(cruces)

# 13. Observaciones influyentes: distancia de Cook > 4/n.
cook <- cooks.distance(modelo)
umbral_cook <- 4 / nrow(train)
influyentes <- data.frame(umbral = umbral_cook, n_influyentes = sum(cook > umbral_cook),
  pct = 100 * mean(cook > umbral_cook), cook_max = max(cook), id_cook_max = train$id[which.max(cook)],
  n_cook_mayor_1 = sum(cook > 1))
guardar(influyentes, "influyentes")
print(influyentes)

# 14. Guardar el modelo y lo necesario para reproducir las predicciones.
# Interpretación: la probabilidad predicha es relativa a ESTA base (≈ 52 % de ataques,
# sin duplicados, ataques sintéticos del laboratorio): se interpretan los odds ratios,
# no prevalencias absolutas. El OR de PCk es por cada desvío del puntaje.
saveRDS(list(modelo = modelo, elegido = elegido, M1 = M1, M2 = M2, acp_entrenamiento = acp_tr,
  idx_train = idx_train, idx_test = idx_test, ids_train = base$id[idx_train],
  ids_test = base$id[idx_test], semilla = 42), "resultados/logistica.rds")
cat("Listo. Modelo", elegido, "| AUC prueba:", coma(as.numeric(auc(roc_te))),
  "IC95 [", coma(ic_auc[1]), ",", coma(ic_auc[3]), "]\n")
```

`R/ejecutar_todo.R` · Ejecución completa del proyecto

```r
# Ejecutar desde la carpeta principal del repositorio.
# Paquetes: install.packages(c("readxl", "psych", "cluster", "ca", "FactoMineR"))
# Las figuras se crean luego con Python; ver python/graficos_parte2.py.
for (archivo in c("00_datos.R", "01_acp.R", "02_af.R", "03_cluster.R", "04_acm.R", "05_integracion.R")) {
  cat("\nEJECUTANDO", archivo, "\n")
  source(paste0("R/", archivo), encoding = "UTF-8")
}
```

B.2. Scripts de Python

`python/graficos_etapa0.py` · Gráficos de la Etapa 0

```python
"""Gráficos pastel de la Etapa 0. Ejecutar después de R/00_datos.R."""
from pathlib import Path

import matplotlib
matplotlib.use("Agg")  # Exportación reproducible: no abre ventanas ni requiere pantalla.
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.ticker import MaxNLocator
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
TABLES = ROOT / "resultados" / "tablas"
FIGURES = ROOT / "resultados" / "figuras"
FIGURES.mkdir(parents=True, exist_ok=True)

def read(name):
    return pd.read_csv(TABLES / f"{name}.csv")

base = pd.read_excel(ROOT / "muestra_estratificada-simulado-de-ataques.xlsx", sheet_name="muestra")
diccionario = read("diccionario_variables")
```

```
conjuntos = read("conjuntos_congelados")
variables = conjuntos.loc[conjuntos.conjunto.eq("X_num"), "variable"].tolist()
crudas = [v.removeprefix("log_") for v in variables]
crudas.insert(2, "dpkts")  # Se describe la respuesta del destino, aunque no entra en X_num.
limites = read("atipicos_limites").set_index("variable")
frecuencias = read("frecuencias_X_cat")
correlaciones = read("correlaciones").set_index("variable")
msa = read("msa_por_variable")
piloto = read("piloto_supuestos").iloc[0]

plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
    "axes.titlesize": 11, "axes.labelsize": 10, "figure.facecolor": "white",
    "axes.facecolor": "white", "axes.spines.top": False, "axes.spines.right": False,
    "text.color": "#283C49", "axes.labelcolor": "#283C49",
    "xtick.color": "#465761", "ytick.color": "#465761"})
PASTEL = ["#C7DCEB", "#D7CCE8", "#CDE2D3", "#F0D4BE", "#EBCDD5"]
EDGE = "#556874"

def guardar(fig, name):
    fig.savefig(FIGURES / f"{name}.png", dpi=200, bbox_inches="tight", facecolor="white")
    plt.close(fig)

def ejes_panel(cantidad=12):
    filas = (cantidad + 3) // 4
    return plt.subplots(filas, 4, figsize=(15, filas * 3.7), layout="constrained")

def nota(fig, texto):
    fig.supxlabel(texto, fontsize=9, color="#465761")

# Se usan las bisagras calculadas por R; bxp evita diferencias de cuartiles por empates.
fig, axes = ejes_panel()
for i, (ax, v) in enumerate(zip(axes.flat, variables)):
    row = limites.loc[v]
    x = base[v].to_numpy()
    extremos = x[(x < row.limite_inferior) | (x > row.limite_superior)]
    stats = {"med": row.mediana, "q1": row.q1_tukey, "q3": row.q3_tukey,
        "whislo": row.bigote_inferior, "whishi": row.bigote_superior, "fliers": extremos}
    ax.bxp([stats], positions=[1], widths=.45, patch_artist=True, manage_ticks=False,
        boxprops={"facecolor": PASTEL[i // 4], "edgecolor": EDGE},
        medianprops={"color": "#283C49", "linewidth": 1.5},
        whiskerprops={"color": EDGE}, capprops={"color": EDGE},
        flierprops={"marker": ".", "markersize": 3, "alpha": .28,
                    "markeredgecolor": EDGE, "markerfacecolor": EDGE})
    ax.set_title(f"{v}\n{int(row.n_atipicos):,} atípicos".replace(",", "."))
    ax.set_xticks([])
    ax.set_ylabel("log(x + 1)")
    ax.grid(axis="y", color="#E5E9EC", linewidth=.5)
fig.suptitle("Control de atípicos · 12 variables seleccionadas", fontsize=17)
nota(fig, "5.000 flujos · bisagras de Tukey ± 1,5 RIC · escalas independientes · se conservan todos los
    ↪ registros")
guardar(fig, "01_boxplots")

# Histogramas: todos los datos, sin recortar colas; bins FD con tope visual de 80.
def histogramas(lista, nombre, titulo, escala):
    fig, axes = ejes_panel(len(lista))
    for i, (ax, v) in enumerate(zip(axes.flat, lista)):
        x = base[v].to_numpy()
        q1, q3 = np.quantile(x, [.25, .75])
        ancho = 2 * (q3 - q1) / np.cbrt(len(x))
        n_bins = int(np.clip(np.ceil(np.ptp(x) / ancho), 10, 80)) if ancho > 0 else 20
        ax.hist(x, bins=n_bins, color=PASTEL[i // 4], edgecolor=EDGE, linewidth=.35)
        unidad = diccionario.set_index("nombre").loc[v, "unidad"]
        ax.set_title(v)
        ax.set_xlabel(unidad.replace(" (ventana", "\n(ventana"))
        ax.set_ylabel("Flujos")
        ax.xaxis.set_major_locator(MaxNLocator(nbins=4))
        ax.ticklabel_format(axis="x", style="sci", scilimits=(-3, 4), useMathText=True)
        ax.grid(axis="y", color="#E5E9EC", linewidth=.5)
    for ax in axes.flat[len(lista):]:
        ax.axis("off")
    fig.suptitle(titulo, fontsize=17)
    nota(fig, f"5.000 flujos · {escala} · rango completo, sin eliminar extremos · intervalos FD limitados a
    ↪ 10–80")
```

```
    guardar(fig, nombre)

histogramas(variables, "02_histogramas_log", "Distribuciones de X_num · escala log(x + 1)", "12 variables seleccionadas")
histogramas(crudas, "03_histogramas_crudos", "Distribuciones originales · comparación con la transformación ", "13 mediciones originales, incluido dpkts")

fig, ax = plt.subplots(figsize=(13, 11), layout="constrained")
cmap = LinearSegmentedColormap.from_list("pastel_divergente", ["#9CBAD4", "#FAFAF7", "#DFB0BC"])
im = ax.imshow(correlaciones.to_numpy(), vmin=-1, vmax=1, cmap=cmap)
labels = [v.removeprefix("log_") for v in variables]
ax.set_xticks(range(12), labels, rotation=50, ha="right")
ax.set_yticks(range(12), labels)
for i in range(12):
    for j in range(12):
        ax.text(j, i, f"{correlaciones.iloc[i, j]:.2f}", ha="center", va="center", fontsize=8, color="#283C49")
fig.colorbar(im, ax=ax, shrink=.75, label="Correlación de Pearson (r)")
ax.set_title("Correlaciones entre las 12 variables de X_num", pad=16, fontsize=16)
nota(fig, "5.000 flujos · todas las variables están en escala log(x + 1) · rango de color fijo: -1 a 1")
guardar(fig, "04_correlaciones")

fig, axes = plt.subplots(2, 4, figsize=(17, 9), layout="constrained")
for i, (ax, v) in enumerate(zip(axes.flat, frecuencias.variable.unique())):
    f = frecuencias[frecuencias.variable.eq(v)].sort_values("frecuencia")
    # Mantener el orden natural de las bandas, aunque sus frecuencias sean similares.
    ordenes = {"duracion_banda": ["larga", "media", "corta"],
               "bytes_origen_banda": ["altos", "medios", "bajos"]}
    if v in ordenes:
        f = f.set_index("categoria").loc[ordenes[v]].reset_index()
    ax.barh(f.categoria, f.porcentaje, color=PASTEL[i % len(PASTEL)], edgecolor=EDGE, linewidth=.5)
    ax.set_xlim(0, 100)
    ax.set_xlabel("Porcentaje de flujos")
    ax.set_title(v)
    ax.grid(axis="x", color="#E5E9EC", linewidth=.5)
    for j, row in enumerate(f.itertuples()):
        ax.text(row.porcentaje + 1, j, f"{row.porcentaje:.1f}%".replace(".", ","), va="center", fontsize=9)
axes.flat[-1].axis("off")
fig.suptitle("Frecuencias de X_cat · propuesta operativa para ACM", fontsize=17)
nota(fig, "5.000 flujos · 4 categóricas existentes + 3 numéricas discretizadas · ninguna modalidad por debajo de 5 %")
guardar(fig, "05_frecuencias_categoricas")

fig, ax = plt.subplots(figsize=(10, 7), layout="constrained")
f = msa.sort_values("msa")
ax.barh(f.variable, f.msa, color=PASTEL[0], edgecolor=EDGE, linewidth=.5)
ax.axvline(.5, color="#7C5363", linestyle="--", linewidth=1.2, label="Umbral MSA = 0,50")
ax.set_xlim(0, 1)
for i, row in enumerate(f.itertuples()):
    ax.text(row.msa + .01, i, f"{row.msa:.3f}", va="center")
ax.set_xlabel("MSA por variable")
ax.set_title(f"Adecuación por variable · KMO global = {piloto.kmo_global:.3f}", fontsize=15)
ax.legend(loc="lower right")
guardar(fig, "06_msa")


print("Se guardaron las seis figuras en resultados/figuras.")
```

`python/graficos_parte2.py` · Gráficos de las Etapas 1 a 4

```
"""Figuras de todas las técnicas. Ejecutar después de R/ejecutar_todo.R."""
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.ticker import MaxNLocator
import numpy as np
import pandas as pd

```

```
ROOT = Path(__file__).resolve().parents[1]
T = ROOT / "resultados/tablas"
F = ROOT / "resultados/figuras"
F.mkdir(exist_ok=True)
def leer(n): return pd.read_csv(T / f"{n}.csv")
base = pd.read_excel(ROOT / "muestra_estratificada-simulado-de-ataques.xlsx", sheet_name="muestra")
vars_num = leer("conjuntos_congelados").query('conjunto == "X_num"').variable.tolist()
ABR = ["dur", "spkts", "sbytes", "dbytes", "sinpkt", "sjit", "synack", "srv_src", "srv_dst", "src_ltm", "
    ↪ src_dport", "dst_sport"]
COL = ["#B9D2E5", "#D1C3E0", "#BCD8C5", "#E6C4A7", "#E5BCC8", "#C7D8D4", "#D8D2B1"]
TRAFICO = dict(zip(["Normal", "Exploits", "Fuzzers", "Reconnaissance", "Otros ataques"], COL))
DARK = "#405361"
CMAP = LinearSegmentedColormap.from_list("suave", ["#94B7D2", "#FCFCF9", "#D9A7B5"])
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 13,
    "axes.spines.top": False, "axes.spines.right": False, "axes.titlesize": 13,
    "text.color": DARK, "axes.labelcolor": DARK, "figure.facecolor": "white"})

def guardar(fig, nombre):
    fig.savefig(F / f"{nombre}.png", dpi=220, bbox_inches="tight", facecolor="white")
    plt.close(fig)

def origen(ax):
    ax.axhline(0, color="#D0D6DB", lw=.7)
    ax.axvline(0, color="#D0D6DB", lw=.7)

def adjust_text(texts, ax, expand=None, arrowprops=None):
    """Colocar etiquetas sin solaparlas, usando solo Matplotlib."""
    ax.set_autoscale_on(False)
    ax.figure.canvas.draw()
    renderer = ax.figure.canvas.get_renderer()
    inv = ax.transData.inverted()
    ocupados = []
    offsets = sorted([(dx,dy) for dx in [0,-20,20,-45,45,-80,80,-130,130,-180,180] for dy in range
    ↪ (-160,161,14)], key=lambda z:z[0]**2+z[1]**2)
    for text in sorted(texts, key=lambda t:t.get_position()[1]):
        original = text.get_position()
        origen = ax.transData.transform(original)
        mejor = None
        for dx,dy in [(0,0)]+offsets:
            text.set_position(inv.transform(origen + [dx,dy]))
            bb = text.get_window_extent(renderer).expanded(1.12,1.3)
            if bb.x0 < ax.bbox.x0 or bb.x1 > ax.bbox.x1 or bb.y0 < ax.bbox.y0 or bb.y1 > ax.bbox.y1:
    ↪ continue
            if any(bb.overlaps(o) for o in ocupados): continue
            mejor = bb; break
        if mejor is None:
            text.set_position(original)
            mejor = text.get_window_extent(renderer)
        ocupados.append(mejor)
        if np.linalg.norm(ax.transData.transform(text.get_position())-origen)>5:
            ax.plot([original[0],text.get_position()[0]],[original[1],text.get_position()[1]],color="#
    ↪ AAB2B9",linewidth=.45)

def dendrogram(link, ax, labels=None, truncate_mode=None, p=25, orientation="top", **kwargs):
    """Dibujar con Matplotlib el árbol Ward ya calculado en R, sin recalcularlo."""
    n = len(link) + 1
    limite = 2 * n - p if truncate_mode else n
    hojas = []
    def ordenar(node):
        if node < n or node < limite:
            hojas.append(node)
        else:
            ordenar(int(link[node - n, 0])); ordenar(int(link[node - n, 1]))
    ordenar(2 * n - 2)
    posiciones = {node: i for i, node in enumerate(hojas)}
    def trazar(node):
        if node in posiciones: return posiciones[node], 0
        row = link[node - n]
        a, ha = trazar(int(row[0])); b, hb = trazar(int(row[1]))
        xx, yy = [a, a, b, b], [ha, row[2], row[2], hb]
        if orientation == "right": ax.plot(yy, xx, color="#8EA9BE", lw=1)
        else: ax.plot(xx, yy, color="#8EA9BE", lw=1)
        return (a + b) / 2, row[2]
```

```
    trazar(2 * n - 2)
    nombres = [(labels[node] if labels else str(node + 1)) if node < n else f"({int(link[node - n, 3])})"
    ↪ for node in hojas]
    if orientation == "right": ax.set_yticks(range(len(hojas)), nombres, fontsize=12)
    else: ax.set_xticks(range(len(hojas)), nombres, fontsize=11)

# Resumen gráfico de datos para el informe; las figuras completas de Etapa 0 se conservan.
fig = plt.figure(figsize=(10, 4.2), layout="constrained")
rejilla = fig.add_gridspec(2, 4, width_ratios=[1, 1, 1.25, 1.25])
axs = np.array([[fig.add_subplot(rejilla[0, 0]), fig.add_subplot(rejilla[0, 1])],
    [fig.add_subplot(rejilla[1, :2]), fig.add_subplot(rejilla[:, 2:])]])
for ax, v, unidad in [(axs[0, 0], "dur", "segundos"), (axs[0, 1], "log_dur", "log(segundos + 1)")]:
    ax.hist(base[v], bins=40, color=COL[0], edgecolor=DARK, lw=.3)
    ax.set(title=v, xlabel=unidad, ylabel="Flujos")
lim = leer("atipicos_limites").set_index("variable")
seleccion = ["log_spkts", "log_sbytes", "log_dbytes"]
stats = []
for v in seleccion:
    r = lim.loc[v]; x = base[v]
    stats.append(dict(label=v.removeprefix("log_"), med=r.mediana, q1=r.q1_tukey,
        q3=r.q3_tukey, whislo=r.bigote_inferior, whishi=r.bigote_superior,
        fliers=x[(x < r.limite_inferior) | (x > r.limite_superior)].to_numpy()))
axs[1, 0].bxp(stats, patch_artist=True, boxprops=dict(facecolor=COL[1], edgecolor=DARK),
    medianprops=dict(color=DARK), flierprops=dict(marker=".", markersize=2, alpha=.2))
axs[1, 0].set(title="Atípicos de volumen", ylabel="log(x + 1)")
corr = leer("correlaciones").set_index("variable")
im = axs[1, 1].imshow(corr, cmap=CMAP, vmin=-1, vmax=1)
axs[1, 1].set_xticks(range(12), ABR, rotation=55, ha="right", fontsize=11)
axs[1, 1].set_yticks(range(12), ABR, fontsize=11)
axs[1, 1].set_title("Correlaciones (12 log_)")
fig.colorbar(im, ax=axs[1, 1], shrink=.7)
guardar(fig, "07_datos_resumen")

# ACP: el biplot usa escalas recíprocas, conservando el producto fila-columna.
eig = leer("acp_autovalores")
scores = leer("acp_puntajes")
r = leer("acp_correlaciones").set_index("variable")
fig, axs = plt.subplots(1, 2, figsize=(9, 3.6), layout="constrained", gridspec_kw={"width_ratios": [1,
    ↪ 1.6]})
axs[0].plot(eig.componente, eig.autovalor, "o-", color="#8AABC5", markeredgecolor=DARK)
axs[0].axhline(1, color=DARK, ls="--", lw=.8)
axs[0].set(title="Codo del ACP", xlabel="Componente", ylabel="Autovalor", xticks=range(1, 13))
ax = axs[1]
sx = np.sqrt(eig.autovalor.iloc[:2].to_numpy())
xy = .5 * scores[["PC1", "PC2"]].to_numpy() / sx
for cat, color in TRAFICO.items():
    sel = scores.tipo_trafico.eq(cat)
    ax.scatter(xy[sel, 0], xy[sel, 1], s=6, color=color, alpha=.4, label=cat, rasterized=True)
texts = []
ax.set_xlim(-2.35, 2.35)
ax.set_ylim(-1.5, 2.1)
for i, v in enumerate(vars_num):
    arrow = 2 * r.loc[v, ["PC1", "PC2"]].to_numpy()
    ax.annotate("", xy=arrow, xytext=(0, 0), arrowprops=dict(arrowstyle="->", color=DARK, lw=.8))
    texts.append(ax.text(arrow[0], arrow[1], ABR[i], fontsize=11))
ext = leer("acp_extremos")
for row in ext.itertuples():
    j = scores.index[scores.id.eq(row.id)][0]
    texts.append(ax.text(*xy[j], str(row.id), fontsize=11, fontweight="bold"))
origen(ax)
ax.set(title="Biplot de individuos y variables", xlabel="CP1 (43,29%)", ylabel="CP2 (16,88%)")
fig.legend(*ax.get_legend_handles_labels(), fontsize=10, loc="outside lower center", ncol=5, markerscale=2)
adjust_text(texts, ax=ax, arrowprops=dict(arrowstyle="-", color="#929BA2", lw=.4))
guardar(fig, "08_acp_codo_biplot")

par = leer("af_paralelo")
fig, ax = plt.subplots(figsize=(9, 2.0), layout="constrained")
ax.plot(par.dimension, par.factor_observado, "o-", color="#8AABC5", label="Factores observados")
ax.plot(par.dimension, par.factor_simulado, "s--", color="#B7A4C9", label="Referencia aleatoria")
ax.axvline(4.5, color=DARK, ls=":", lw=.8)
ax.set(xlabel="Número de factor", ylabel="Autovalor reducido", title="Análisis paralelo de factores · 100
    ↪ repeticiones", xticks=range(1, 13))
ax.legend(fontsize=12)
```

```
guardar(fig, "09_af_paralelo")

def linkage_r(nombre, n):
    d = leer(nombre); counts = np.zeros(n - 1); rows = []
    for i, row in enumerate(d.itertuples()):
        a = -int(row.izquierda) - 1 if row.izquierda < 0 else n + int(row.izquierda) - 1
        b = -int(row.derecha) - 1 if row.derecha < 0 else n + int(row.derecha) - 1
        count = (1 if a < n else counts[a - n]) + (1 if b < n else counts[b - n])
        counts[i] = count
        rows.append([a, b, row.altura, count])
    return np.array(rows)

metricas = leer("cluster_metricas")
k = int(leer("cluster_comparaciones").iloc[0].k)
fig, axs = plt.subplots(1, 2, figsize=(10, 2.8), layout="constrained")
for ax, pref, titulo, ylabel in [(axs[0], "wss", "Codo", "Suma de cuadrados interna"),
    (axs[1], "silhouette", "Silhouette promedio", "Silhouette")]:
    ax.plot(metricas.k, metricas[f"{pref}_ward"], "s--", color="#B7A4C9", label="Ward")
    ax.plot(metricas.k, metricas[f"{pref}_kmeans"], "o-", color="#8AABC5", label="K-means")
    ax.axvline(k, color=DARK, lw=.7, ls=":")
    ax.set(title=titulo, xlabel="k", ylabel=ylabel, xticks=range(2, 9))
    ax.legend(fontsize=12)
guardar(fig, "10_cluster_seleccion")

fig, ax = plt.subplots(figsize=(10, 2.6), layout="constrained")
link = linkage_r("cluster_dendrograma", len(base))
dendrogram(link, truncate_mode="lastp", p=15, ax=ax, link_color_func=lambda _: "#8EA9BE", leaf_font_size=8)
ax.axhline((link[-k, 2] + link[-k + 1, 2]) / 2, color=DARK, ls="--", lw=.8)
ax.set(title="Ward.D2 · últimos 15 nodos del árbol de 5.000 flujos", xlabel="Nodos agregados (tamaño entre
    ↪ paréntesis)", ylabel="Altura")
guardar(fig, "11_cluster_dendrograma")

fig, axs = plt.subplots(1, 2, figsize=(10, 3.8), layout="constrained", gridspec_kw={"width_ratios": [1.35,
    ↪ 1]})
perfil = leer("cluster_medias_estandarizadas").set_index("cluster")
im = axs[0].imshow(perfil, cmap=CMAP, vmin=-3, vmax=3, aspect="auto")
axs[0].set_xticks(range(12), ABR, rotation=55, ha="right", fontsize=11)
axs[0].set_yticks(range(k), [f"C{i}" for i in range(1, k + 1)])
axs[0].set_title("Medias z por cluster")
for i in range(k):
    for j in range(12): axs[0].text(j, i, f"{perfil.iloc[i,j]:.1f}", ha="center", va="center", fontsize=12)
fig.colorbar(im, ax=axs[0], shrink=.65, label="z (colores saturan en ±3)")
dendrogram(linkage_r("cluster_dendrograma_variables", 12), labels=ABR, orientation="right",
    ax=axs[1], link_color_func=lambda _: "#8EA9BE", leaf_font_size=9)
axs[1].set(title="Ward de variables", xlabel="Distancia")
guardar(fig, "12_cluster_perfiles_variables")

fig, ax = plt.subplots(figsize=(8, 3.1), layout="constrained")
texts = []
coords_acs = []
for name, color, marker in [("acs_filas", COL[0], "o"), ("acs_columnas", COL[4], "s")]:
    d = leer(name)
    coords_acs.append(d.iloc[:, 1:3].to_numpy())
    ax.scatter(d.iloc[:,1], d.iloc[:,2], color=color, edgecolor=DARK, s=50, marker=marker,
        label="Tipo de tráfico" if name.endswith("filas") else "Estado")
    texts.extend(ax.text(row.iloc[1], row.iloc[2], row.iloc[0], fontsize=12) for _, row in d.iterrows())
coords_acs = np.vstack(coords_acs)
ax.set_xlim(coords_acs[:, 0].min() - .2, coords_acs[:, 0].max() + .2)
ax.set_ylim(coords_acs[:, 1].min() - .2, coords_acs[:, 1].max() + .2)
origen(ax)
ax.set(xlabel="Dimensión 1", ylabel="Dimensión 2", title="ACS · tipo de tráfico y estado")
ax.legend(fontsize=12)
adjust_text(texts, ax=ax, arrowprops=dict(arrowstyle="-", color="#AAB2B9", lw=.5))
guardar(fig, "13_acs_mapa")

e = leer("acm_autovalores")
fig, axs = plt.subplots(1, 2, figsize=(10, 2.1), layout="constrained")
axs[0].plot(e.dimension, e.autovalor, "o-", color="#8AABC5")
axs[0].axhline(1/7, color=DARK, ls="--", lw=.8)
axs[0].set(title="Autovalores crudos del ACM", xlabel="Dimensión", ylabel="Autovalor")
axs[1].bar(e.dimension.iloc[:6], e.porcentaje_ajustado.iloc[:6], color=COL[1], edgecolor=DARK, linewidth
    ↪ =.5)
axs[1].set(title="Inercia ajustada (ca::mjca)", xlabel="Dimensión", ylabel="Porcentaje ajustado")
```

```
guardar(fig, "14_acm_inercia")

co = leer("acm_categorias_coordenadas")
variables_cat = co.categoria.str.split(":").str[0].unique().tolist()
prefijos = dict(zip(variables_cat, ["Tr", "Pr", "Sv", "Es", "Rp", "Du", "By"]))
marcas = ["o", "s", "^", "D", "v", "P", "X"]
fig, ax = plt.subplots(figsize=(9, 4.9), layout="constrained")
texts = []
for i, var in enumerate(variables_cat):
    d = co[co.categoria.str.startswith(var + ":")]
    ax.scatter(d["Dim.1"], d["Dim.2"], color=COL[i], edgecolor=DARK, linewidth=.6,
        marker=marcas[i], s=45, label=f"{prefijos[var]}: {var}")
    for row in d.itertuples(index=False, name=None):
        lab = row[0].split(":", 1)[1].replace("Otros ataques", "Otros at.").replace("Reconnaissance", "
    ↪ Recon.")
        texts.append(ax.text(row[1], row[2], prefijos[var] + ":" + lab, fontsize=12))
ax.margins(.22)
origen(ax)
ax.set(title="ACM · categorías activas", xlabel="Dimensión 1 · 27,42% cruda", ylabel="Dimensión 2 · 14,03%
    ↪ cruda")
fig.supxlabel("Tr tráfico · Pr protocolo · Sv servicio · Es estado · Rp respuesta · Du duración · By bytes",
    ↪ fontsize=10)
adjust_text(texts, ax=ax, expand=(1.1, 1.2), arrowprops=dict(arrowstyle="-", color="#AAB2B9", lw=.5))
guardar(fig, "15_acm_categorias")

fig, axs = plt.subplots(1, 2, figsize=(10, 3.5), layout="constrained")
ind = leer("acm_individuos")
su = leer("acm_cluster_suplementario")
for i in range(1, k + 1):
    d = ind[ind.cluster.eq(i)]
    axs[0].scatter(d["Dim.1"], d["Dim.2"], s=5, alpha=.15, color=COL[i - 1], rasterized=True)
    r = su[su.cluster.eq(i)].iloc[0]
    axs[0].scatter(r["Dim.1"], r["Dim.2"], color=COL[i - 1], edgecolor=DARK, marker="D", s=65)
    axs[0].annotate(f"C{i}", (r["Dim.1"], r["Dim.2"]), xytext=(5, 4), textcoords="offset points", fontsize
    ↪ =12)
origen(axs[0])
axs[0].set(title="Clusters suplementarios", xlabel="Dimensión 1", ylabel="Dimensión 2")
eta = leer("acm_eta2")
y = np.arange(len(eta))
axs[1].barh(y - .18, eta["Dim.1"], height=.35, color=COL[0], edgecolor=DARK, lw=.4, label="Dim 1")
axs[1].barh(y + .18, eta["Dim.2"], height=.35, color=COL[1], edgecolor=DARK, lw=.4, label="Dim 2")
axs[1].set_yticks(y, eta.variable, fontsize=12)
axs[1].set(xlim=(0, 1), xlabel="eta$^2$", title="Discriminación por variable")
axs[1].legend(fontsize=12)
guardar(fig, "16_acm_clusters_eta")
print("Figuras 07 a 16 generadas con Matplotlib.")
```

`python/graficos_logistica.py` · Gráficos de la regresión logística

```
"""Figuras 17 y 18 de la regresión logística. Ejecutar después de R/06_logistica.R."""
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
T = ROOT / "resultados/tablas"
F = ROOT / "resultados/figuras"
F.mkdir(exist_ok=True)
def leer(n): return pd.read_csv(T / f"log_reg_{n}.csv")

# Mismo estilo que graficos_parte2.py
COL = ["#B9D2E5", "#D1C3E0", "#BCD8C5", "#E6C4A7", "#E5BCC8", "#C7D8D4", "#D8D2B1"]
DARK = "#405361"
CMAP = LinearSegmentedColormap.from_list("suave", ["#FCFCF9", "#94B7D2"])
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 13,
    "axes.spines.top": False, "axes.spines.right": False, "axes.titlesize": 13,
    "text.color": DARK, "axes.labelcolor": DARK, "figure.facecolor": "white"})
```

```

def guardar(fig, nombre):
    fig.savefig(F / f"{nombre}.png", dpi=220, bbox_inches="tight", facecolor="white")
    plt.close(fig)

roc = leer("roc_curva")
auc = leer("auc").iloc[0]
conf = leer("confusion")
met = leer("metricas").iloc[0]

# Figura 17: ROC en prueba + matriz de confusión (umbral 0,5)
fig, axs = plt.subplots(1, 2, figsize=(10, 4.2), layout="constrained")
ax = axs[0]
ax.plot([0, 1], [0, 1], ls="--", color="#AAB2B9", lw=1, label="Azar (AUC = 0,500)")
ax.plot(roc.fpr, roc.tpr, color="#6F9BBC", lw=2.2,
        label=f"Modelo (AUC = {auc.AUC:.3f})".replace(".", ","))
ax.fill_between(roc.fpr, roc.tpr, alpha=.15, color=COL[0])
punto = (1 - met.especificidad, met.sensibilidad)
ax.scatter(*punto, s=55, color="#D9A7B5", edgecolor=DARK, zorder=3, label="Umbral 0,5")
ax.set(xlabel="1 - especificidad (falsos positivos)", ylabel="Sensibilidad (verdaderos positivos)",
       xlim=(0, 1), ylim=(0, 1.01), aspect="equal")
ax.legend(loc="lower right", fontsize=11, frameon=False)

ax = axs[1]
m = np.array([[conf.query('real=="normal" and predicho=="normal"').n.iloc[0],
               conf.query('real=="normal" and predicho=="ataque"').n.iloc[0]],
              [conf.query('real=="ataque" and predicho=="normal"').n.iloc[0],
               conf.query('real=="ataque" and predicho=="ataque"').n.iloc[0]]])
pct = m / m.sum(axis=1, keepdims=True)
ax.imshow(pct, cmap=CMAP, vmin=0, vmax=1)
for i in range(2):
    for j in range(2):
        ax.text(j, i, f"{m[i, j]}\n({100 * pct[i, j]:.1f}%)".replace(".", ","), ha="center", va="center",
                fontsize=14, color=DARK, fontweight="bold" if i == j else "normal")
ax.set_xticks([0, 1], ["Normal", "Ataque"])
ax.set_yticks([0, 1], ["Normal", "Ataque"])
ax.set(xlabel="Clase predicha", ylabel="Clase real")
for s in ax.spines.values():
    s.set_visible(False)
ax.tick_params(length=0)
guardar(fig, "17_logistica_roc_confusion")

# Figura 18: forest plot de OR (escala log) + calibración por deciles
co = leer("coeficientes").set_index("termino")
orx = leer("or_por_desvio").set_index("termino")
nombres = {"PC1": "PC1 (por desvío)", "PC2": "PC2 (por desvío)", "PC3": "PC3 (por desvío)",
           "proto_gudp": "Protocolo: udp (vs tcp)", "proto_gotros": "Protocolo: otros (vs tcp)",
           "service_gdns": "Servicio: dns (vs ninguno)", "service_ghttp": "Servicio: http (vs ninguno)",
           "service_gotros": "Servicio: otros (vs ninguno)",
           "state_gCON_otros": "Estado: CON_otros (vs FIN)", "state_gINT": "Estado: INT (vs FIN)"}
filas = []
for t in nombres:
    if t not in co.index: continue
    if t in orx.index:
        r = orx.loc[t]; filas.append((nombres[t], r.OR_por_desvio, r.OR_desvio_IC95_inf, r.
    ↪ OR_desvio_IC95_sup, t))
    else:
        r = co.loc[t]; filas.append((nombres[t], r.OR, r.OR_IC95_inf, r.OR_IC95_sup, t))
filas = filas[::-1]
fig, axs = plt.subplots(1, 2, figsize=(10.5, 4.6), layout="constrained", gridspec_kw={"width_ratios":
    ↪ [1.25, 1]})
ax = axs[0]
y = np.arange(len(filas))
for i, (n, o, lo, hi, t) in enumerate(filas):
    color = "#6F9BBC" if t.startswith("PC") else "#C9899B"
    ax.plot([lo, hi], [i, i], color=color, lw=2)
    ax.scatter(o, i, color=color, edgecolor=DARK, s=40, zorder=3)
ax.axvline(1, color=DARK, lw=.9, ls="--")
ax.set_xscale("log")
ax.set_xticks([.05, .2, 1, 5, 20, 100, 400], ["0,05", "0,2", "1", "5", "20", "100", "400"])
ax.minorticks_off()
ax.set_yticks(y, [f[0] for f in filas], fontsize=11)
ax.set(xlabel="Odds ratio e IC 95 % (escala log)")
```

```

ax = axs[1]
cal = leer("calibracion")
ax.plot([0, 1], [0, 1], ls="--", color="#AAB2B9", lw=1, label="Calibración perfecta")
ax.plot(cal.p_media, cal.prop_observada, "o-", color="#6F9BBC", markeredgecolor=DARK, label="Deciles en prueba")
ax.set(xlabel="Probabilidad media predicha", ylabel="Proporción observada de ataques",
       xlim=(0, 1), ylim=(0, 1.02), aspect="equal")
ax.legend(loc="upper left", fontsize=11, frameon=False)
guardar(fig, "18_logistica_or_calibracion")
print("Figuras 17 y 18 generadas con Matplotlib.")
```

`python/verificar_resultados.py` · Verificación independiente de resultados

```
"""Comprobación independiente de los resultados de R con NumPy y pandas."""
from pathlib import Path
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
T = ROOT / "resultados/tablas"
def leer(n): return pd.read_csv(T / f"{n}.csv")
base = pd.read_excel(ROOT / "muestra_estratificada-simulado-de-ataques.xlsx", sheet_name="muestra")
assert base.shape == (5000, 68)
variables = leer("conjuntos_congelados").query('conjunto == "X_num"').variable.tolist()
x = base[variables].to_numpy()
z = (x - x.mean(axis=0)) / x.std(axis=0, ddof=1)
R = np.corrcoef(x, rowvar=False)
eig = np.linalg.eigvalsh(R)[::-1]
np.testing.assert_allclose(eig, leer("acp_autovalores").autovalor, atol=1e-12)
np.testing.assert_allclose(R, leer("correlaciones").iloc[:, 1:], atol=1e-12)
inversa = np.linalg.inv(R)
anti = inversa / np.sqrt(np.outer(inversa.diagonal(), inversa.diagonal()))
r2 = R**2; q2 = anti**2
np.fill_diagonal(r2, 0); np.fill_diagonal(q2, 0)
np.testing.assert_allclose(r2.sum() / (r2.sum() + q2.sum()), leer("piloto_supuestos").kmo_global.iloc[0])

asignaciones = leer("cluster_asignaciones")
np.testing.assert_array_equal(asignaciones.id, base.id)
wss = 0
for g in sorted(asignaciones.cluster.unique()):
    grupo = z[asignaciones.cluster.eq(g)]
    wss += ((grupo - grupo.mean(axis=0))**2).sum()
np.testing.assert_allclose(wss, leer("cluster_metricas").query('k == 6').wss_kmeans.iloc[0])
assert leer("cluster_tamanos").n.sum() == 5000
af = leer("af_cargas_completas")
# factanal calcula unicidades por optimización; se admite su tolerancia numérica.
np.testing.assert_allclose((af[["F1", "F2", "F3", "F4"]]**2).sum(axis=1), af.comunalidad, atol=1e-4)
assert (af.unicidad > 0).all()
assert leer("af_diagnostico_minres").heywood.iloc[0]

# Rehacer el ACM indicador y el ajuste de Greenacre sin FactoMineR ni ca.
q = base[["tipo_trafico", "proto_g", "service_g", "state_g"]].copy()
q["respuesta_destino"] = np.where(base.dpkts.eq(0), "sin_respuesta", "con_respuesta")
c = leer("cortes_discretizacion").set_index("variable")
q["duracion_banda"] = pd.cut(base.dur, [-np.inf, c.loc["dur", "corte_1"], c.loc["dur", "corte_2"], np.inf],
    labels=["corta", "media", "larga"])
q["bytes_origen_banda"] = pd.cut(base.sbytes, [-np.inf, 564, 1580, np.inf], labels=["bajos", "medios", "altos"])
A = pd.get_dummies(q, dtype=float).to_numpy()
n, Q, J = len(q), len(q.columns), A.shape[1]
P = A / (n * Q)
cm = P.sum(axis=0)
S = (P - cm / n) / np.sqrt(cm / n)
lamb = np.linalg.svd(S, compute_uv=False)[:J - Q]**2
acm = leer("acm_autovalores")
np.testing.assert_allclose(lamb, acm.autovalor, atol=1e-11)
ajustados = np.where(lamb > 1 / Q, (Q / (Q - 1) * (lamb - 1 / Q))**2, 0)
total = Q / (Q - 1) * (np.sum(lamb**2) - (J - Q) / Q**2)
np.testing.assert_allclose(ajustados, acm.autovalor_ajustado, atol=1e-11)
np.testing.assert_allclose(100 * ajustados / total, acm.porcentaje_ajustado, atol=1e-10)
```

```
t = leer("acs_contingencia").iloc[:, 1:].to_numpy()
esperados = np.outer(t.sum(axis=1), t.sum(axis=0)) / t.sum()
np.testing.assert_allclose(((t - esperados)**2 / esperados).sum(), leer("acs_asociacion").chi2.iloc[0])
assert (leer("frecuencias_X_cat").groupby("variable").frecuencia.sum() == 5000).all()
print("Verificación correcta: ACP, KMO, WSS, AF, ACS y ACM con ajuste de inercia.")
```